EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

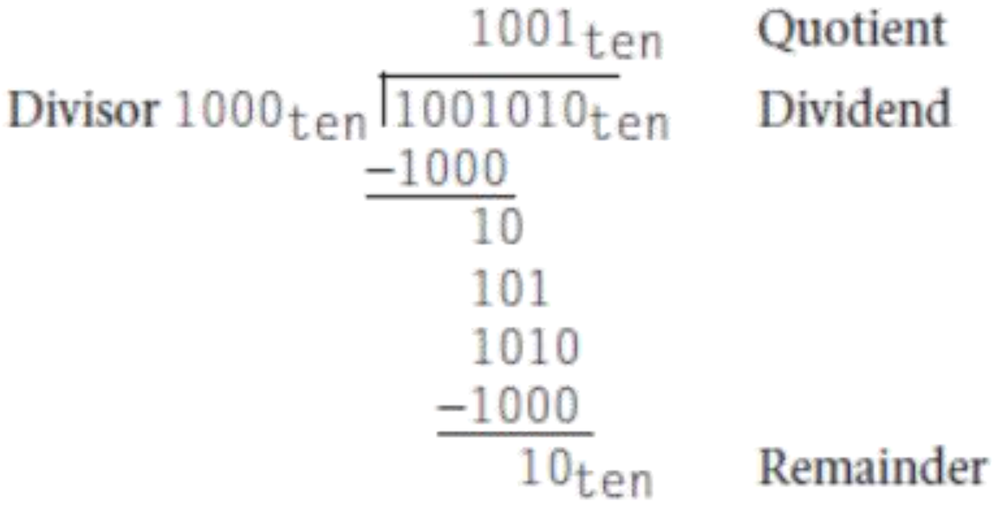
National University of Science and Technology (NUST)

Week # 8_OLE

Division

- Relatively easy in binary
  - Can be accomplished via shifting and addition/subtraction
  - Quotient can only be 1 or 0 i.e. **Either** directly subtract divisor from the dividend and insert 1 in quotient **OR** insert 0 in the quotient
  - **Dividend = Divisor x Quotient + Remainder**

- Division of two **n-bit** numbers will generate two results (remainder & quotient), each up **to n-bits**
- Will look into 2 versions of division algorithm



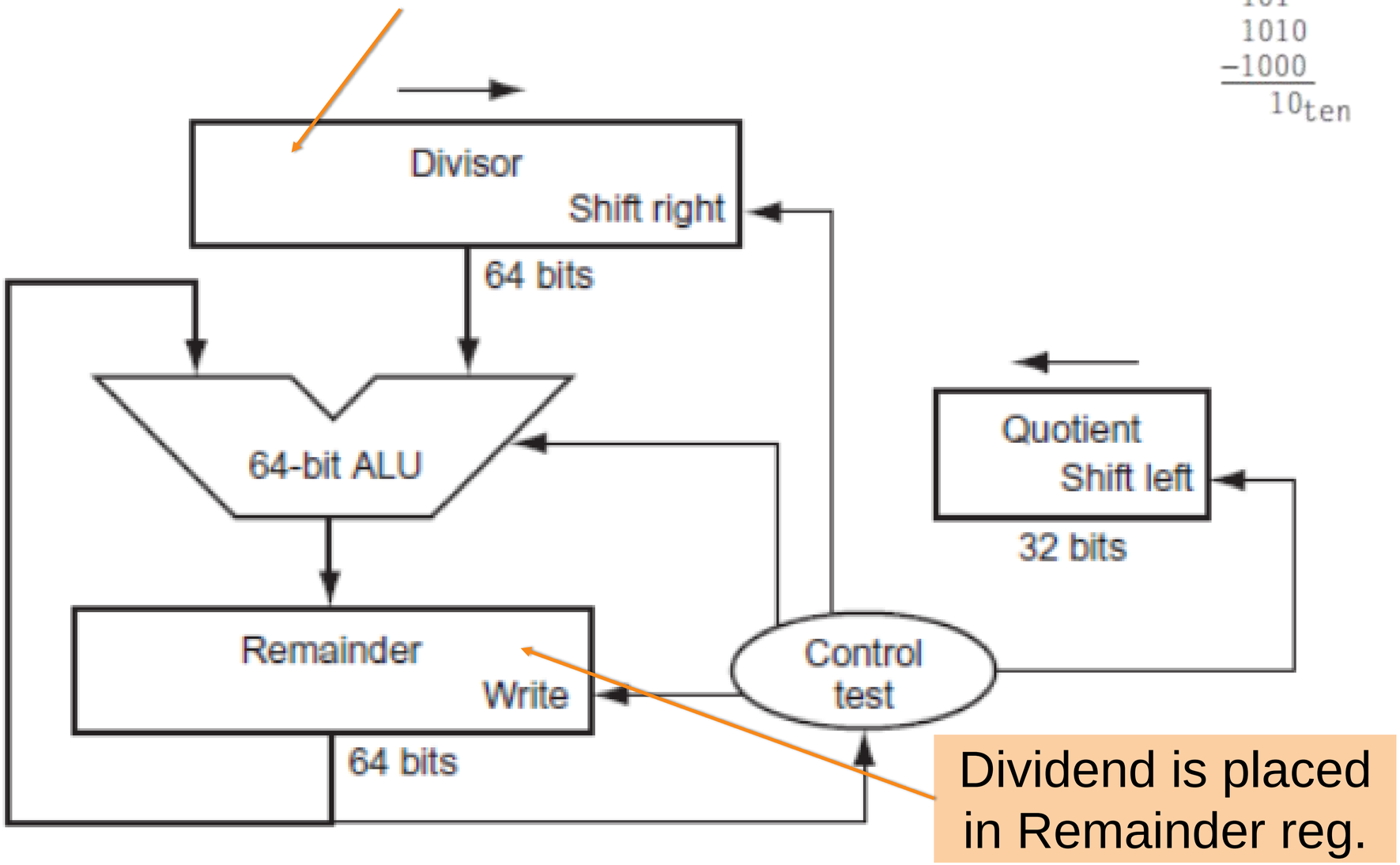
- Negative numbers: more difficult
- There are better techniques, we won't look at them

Division Hardware

- **First version**

Divisor in left half of Divisor reg.

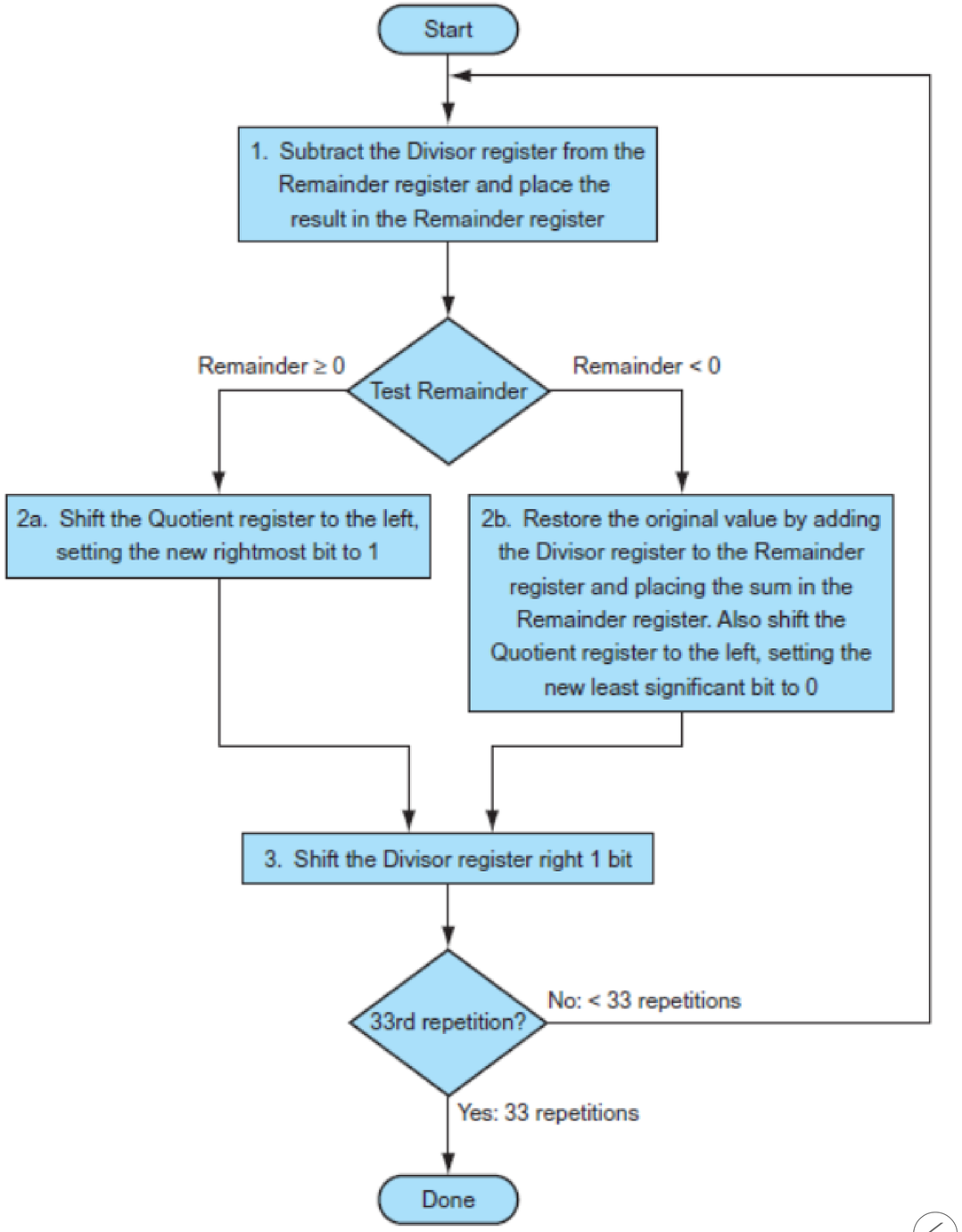
$$
\begin{array}{rll}
 & 1001_{ten} & \text{Quotient} \\
\text{Divisor } 1000_{ten} & \overline{)\,1001010_{ten}} & \text{Dividend} \\
 & -1000 & \\
 & 10 & \\
 & 101 & \\
 & 1010 & \\
 & -1000 & \\
 & 10_{ten} & \text{Remainder}
\end{array}
$$



Dividend is placed in Remainder reg.

First Division Algorithm

- **Three basic steps for each bit**
  - Divisor Subtraction
  - Left shift Quotient
  - Right shift Divisor
- **33 repetitions to get result**
- **Require almost 100 clock cycles**
- **Division is Slow**
  - Avoid as much as possible, use shift operation directly for division by powers of 2.



Example: Division Algorithm

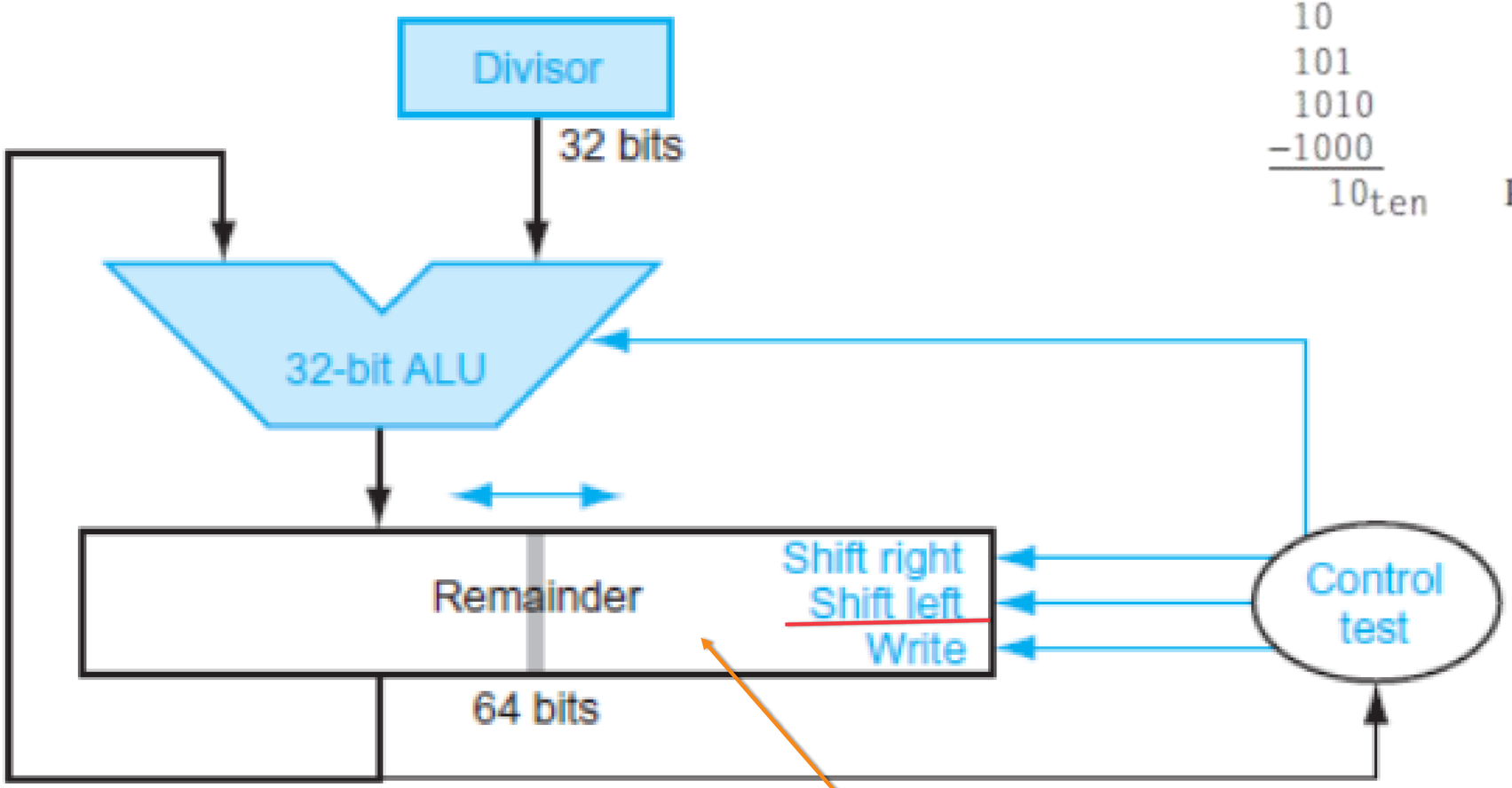
- **Division example using previous algorithm (7 / 2)**

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem – Div	0000	0010 0000	1110 0111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem – Div	0000	0001 0000	1111 0111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem – Div	0000	0000 1000	1111 1111
	2b: Rem < 0 ⇒ +Div, sll Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem – Div	0000	0000 0100	0000 0011
	2a: Rem ≥ 0 ⇒ sll Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem – Div	0001	0000 0010	0000 0001
	2a: Rem ≥ 0 ⇒ sll Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Division Hardware– Refined Version

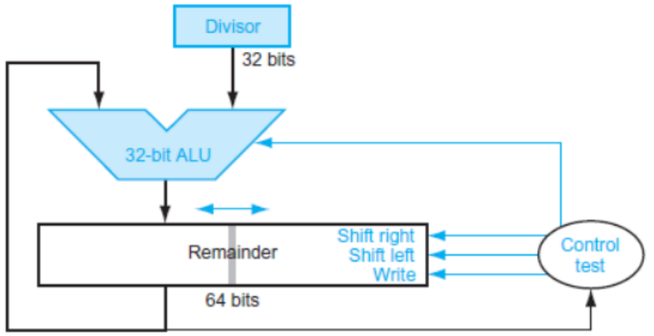
$$
\begin{array}{rll}
 & 1001_{ten} & \text{Quotient} \\
\text{Divisor } 1000_{ten} & \overline{)\,1001010_{ten}} & \text{Dividend} \\
 & -1000 & \\
 & \quad 10 & \\
 & \quad 101 & \\
 & \quad 1010 & \\
 & \quad -1000 & \\
 & \quad 10_{ten} & \text{Remainder}
\end{array}
$$

Final Version



Dividend in Rem. reg.

Refined Algorithm



Start

1. Shift the Remainder reg. left by 1 bit

2. Subtract the Divisor register from the **left half of the Remainder register** and place the result in the left half of the remainder register

Test Remainder

Rem. >= 0

Rem. < 0

3a. Shift the **Remainder** register to the left, setting the new rightmost bit to 1

3b. Restore the original value by adding the Divisor register to **the left half of the Rem. Reg**. and placing the result in the left half of the Rem. Reg. Also, Shift the **Rem. Reg.** to the left, setting the new rightmost bit to 0

32^{nd} repetitions

No <32 iterations

Yes

Done. Shift **left half** of Remainder reg. to the **right by 1 bit**

Example– Refined Version

Iteration	Divisor	Divide algorithm	
		Step	Remainder
0	0010	Initial values	0000 0111
	0010	Shift Rem left 1	0000 1110
1	0010	2: Rem = Rem - Div	1110 1110
	0010	3b: Rem < 0 ⇒ + Div, sll R, R0 = 0	0001 1100
2	0010	2: Rem = Rem - Div	1111 1100
	0010	3b: Rem < 0 ⇒ + Div, sll R, R0 = 0	0011 1000
3	0010	2: Rem = Rem - Div	0001 1000
	0010	3a: Rem ≥ 0 ⇒ sll R, R0 = 1	0011 0001
4	0010	2: Rem = Rem - Div	0001 0001
	0010	3a: Rem ≥ 0 ⇒ sll R, R0 = 1	0010 0011
Done	0010	shift left half of Rem right 1	0001 0011

Signed Division

- **Convert operands to positive numbers**
- **Remember Signs of original digits**
- **After Division, add appropriate signs**
  - Quotient: **-ve if** Divisor and Dividend **signs differ** else +ve
  - Remainder: same sign as of **Dividend**

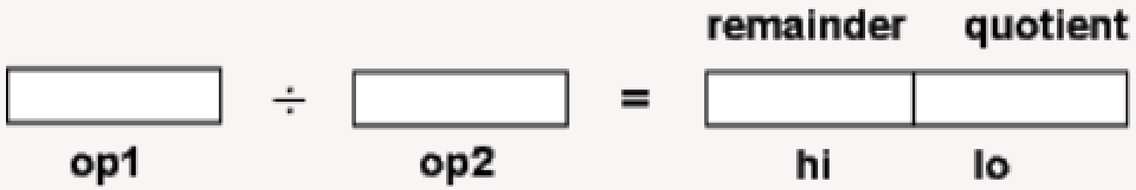
Faster Division

- Unlike faster multiplication, we can't implement faster Division just by adding more hardware / ALU's
  - At each step, the sign of difference or subtraction is needed to execute next step

- **There are techniques exists for faster division,**
  - normally they predict quotient bits, typically 4 bits e.g. *SRT technique*
  - mispredictions handled in subsequent steps

Division in MIPS

- *Instructions:* **div, divu**
- *Registers:* **Hi – Lo,** will hold the **Remainder - Quotient**




```
div     s,t          #  lo <- s div t
                     #  hi <- s mod t
                     #  operands are two's complement
divu    s,t          #  lo <- s div t
                     #  hi <- s mod t
                     #  operands are unsigned
```

Multiplication & Division in MIPS

- *Summary*

multiply	mult	$s2,$s3	Hi, Lo = $s2 x $s3	64-bit signed product in Hi, Lo
multiply unsigned	multu	$s2,$s3	Hi, Lo = $s2 x $s3	64-bit unsigned product in Hi, Lo
divide	div	$s2,$s3	Lo = $s2 / $s3, Hi = $s2 mod $s3	Lo = quotient, Hi = remainder
divide unsigned	divu	$s2,$s3	Lo = $s2 / $s3, Hi = $s2 mod $s3	Unsigned quotient and remainder
move from Hi	mfhi	$s1	$s1 = Hi	Used to get copy of Hi
move from Lo	mflo	$s1	$s1 = Lo	Used to get copy of Lo